

NYP

Uygulama 6

Kitap Sınıfı (Constructor)

- Kitap adında bir sınıf oluşturun.
- Alanlar: String baslik, String yazar, int sayfaSayisi, double fiyat
- Constructor: Tüm alanları parametre olarak alsın.
- İkinci constructor: Sadece baslik ve yazar alsın, sayfaSayisi=0, fiyat=0 olsun.
- Method: void bilgiYazdir() → Tüm bilgileri yazdırsın.
- Method: double indirimliF(double yuzde) → İndirimli fiyatı döndürsün.
- main'de her iki constructor'ı da kullanarak kitap oluşturun.

```
public class Main {  
    public static void main(String[] args) {  
        Kitap k1 = new Kitap("Sefiller", "Victor Hugo", 1400, 85.0);  
        Kitap k2 = new Kitap("Yeni Roman", "Anonim");  
  
        k1.bilgiYazdir();  
        // Sefiller - Victor Hugo (1400 sayfa, 85.0 TL)  
  
        k2.bilgiYazdir();  
        // Yeni Roman - Anonim (0 sayfa, 0.0 TL)  
  
        System.out.println("Indirimli: "  
            + k1.indirimliFiyat(20) + " TL");  
        // Indirimli: 68.0 TL  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Kitap k1 = new Kitap("Sefiller", "Victor Hugo", 1400, 85.0);  
        Kitap k2 = new Kitap("Yeni Roman", "Anonim");  
  
        k1.bilgiYazdir();  
        // Sefiller - Victor Hugo (1400 sayfa, 85.0 TL)  
  
        k2.bilgiYazdir();  
        // Yeni Roman - Anonim (0 sayfa, 0.0 TL)  
  
        System.out.println("Indirimli: "  
            + k1.indirimliFiyat(20) + " TL");  
        // Indirimli: 68.0 TL  
    }  
}
```

ArrayList – Temel Kullanım

```
import java.util.ArrayList;

ArrayList<String> isimler = new ArrayList<>();
isimler.add("Ali");           // Ekleme
isimler.get(0);               // Erişim → "Ali"
isimler.size();               // Boyut → 1
isimler.remove(0);            // Silme
```

```
// ★ Nesnelerle kullanım – aynı mantık!
ArrayList<Ogrenci> liste = new ArrayList<>();
liste.add(new Ogrenci("Ali", 101, 85.5));
liste.add(new Ogrenci("Ayse", 102, 92.0));

for (Ogrenci o : liste) {     // Döngü ile erişim
    System.out.println(o.ad + " - " + o.ortalama);
}

Ogrenci ilk = liste.get(0);    // Index ile erişim
```

Dikkat: ArrayList<int> olmaz → ArrayList<Integer> kullan! (Wrapper class)

Arama ve Filtreleme Kalıpları

```
// ★ İsme göre arama
public static Ogrenci ogrenciBul(ArrayList<Ogrenci> liste, String ad) {
    for (Ogrenci o : liste) {
        if (o.ad.equals(ad)) return o;
    }
    return null; // Bulunamadı
}

// ★ Koşula göre filtreleme (ortalama >= 80 olanlar)
public static ArrayList<Ogrenci> basarililar(ArrayList<Ogrenci> liste) {
    ArrayList<Ogrenci> sonuc = new ArrayList<>();
    for (Ogrenci o : liste) {
        if (o.ortalama >= 80) sonuc.add(o);
    }
    return sonuc;
}

// ★ En yüksek ortalamayı bulma
public static Ogrenci enBasarili(ArrayList<Ogrenci> liste) {
    Ogrenci en = liste.get(0);
    for (Ogrenci o : liste) {
        if (o.ortalama > en.ortalama) en = o;
    }
    return en;
}
```

Ürün Kataloğu

- Urun sınıfı: String ad, String kategori, double fiyat, int stok (Constructor ile)
- main'de ArrayList<Urun> oluşturup en az 5 ürün ekleyin.
- Tüm ürünleri listeleyen bir method yazın.
- Belirli bir kategorideki ürünleri filtreleyen method yazın.
- Fiyat aralığına göre (min-max) ürün arayan method yazın.
- Stokta olan ürünlerin toplam değerini (fiyat × stok) hesaplayın.